# HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

# BÁO CÁO TUẦN 1

**Đề tài: Bài toán nhận diện biển số xe**

| | |
|---|---|
| **Giảng viên hướng dẫn:** | Kim Ngọc Bách |
| **Nhóm lớp** | 11 |
| **Họ và tên sinh viên** | Đinh Hoàng Vũ |
| **Mã sinh viên** | B23DCCN941 |

***Hà Nội – 2026***

**Mục lục**


**I. Neural Networks** ..... 3

**1. Mạng Neural Network là gì?** ..... 3

**2. Cách hoạt động của Neural Network** ..... 3

**2.1. Forward Propagation (Lan truyền xuôi)** ..... 3

**2.2. Back Propagation (Lan truyền ngược)** ..... 4

**2.3. Iteration** ..... 4

**II. Convolution Neural Network** ..... 4

**1. Convolution Neural Network là gì?** ..... 4

**2. Các thành phần chính của CNN** ..... 5

**2.1. Input Layer** ..... 5

**2.2. Convolutional Layer (Lớp tích chập)** ..... 5

**2.3. Activation Layer** ..... 5

**2.4. Pooling Layer** ..... 5

**2.5. Flattening** ..... 5

**2.6. Fully Connected Layer** ..... 6

**2.7. Oupur Layer** ..... 6

**3. Convolutional Layers hoạt động như thế nào** ..... 6

**4. Triển khai một bộ phân loại ảnh dựa trên CNN sử dụng thư viện PyTorch** ..... 6

**III. Object Detection** ..... 6

**1. Bounding Box Detection** ..... 6

**1.1. Bounding Box là gì?** ..... 6

**1.2. Triển khai một ví dụ của Bounding Box Detection sử dụng thư viện PyTorch** ..... 7

**2. Intersection over Union (IuO)** ..... 7

**3. Non-Max Suppression** ..... 8

BÁO CÁO TUẦN LẦN 1 HỌC PHẦN THỰC TẬP CƠ SỞ

TÌM HIỂU CÁC LÝ THUYẾT CẦN THIẾT CHO DỰ ÁN

# I. Neural Networks

## 1. Mạng Neural Network là gì?

**Neural Network** là những model học máy bắt chước những chức năng phức tạp của bộ não con người. Những mô hình này bao gồm các node hoặc neuron được kết nối với nhau để xử lý dữ liệu, học các mẫu và kích hoạt các nhiệm vụ như nhận diện mẫu và ra quyết định

Neural Network có khả năng học và nhận diện các mẫu từ dữ liệu mà không cần các quy luật định nghĩa trước. Các mạng này được xây dựng dựa trên các thành phần chính:

- **Neurons:** Thành phần cơ bản để nhận các input, mỗi neuron bị chi phối bởi một ngưỡng và một hàm kích hoạt.
- **Connections:** Kết nối giữa các neuron truyền thông tin được điều chỉnh bởi weights và bias.
- **Weights và biases:** Là các tham số định nghĩa độ mạnh và sự ảnh hưởng của các kết nối
- **Propagation Functions:** Chức năng lan truyền – cơ chết giúp xử lý và truyền dữ liệu qua các neural
- **Learning Rule:** Quy tắc học – Là phương pháp điều chỉnh weights và biases thường xuyên để cải thiện độ chính xác

## 2. Cách hoạt động của Neural Network

### 2.1.Forward Propagation (Lan truyền xuôi)

Khi dữ liện là input đi vào mạng, nó đi qua các mạng theo chiều xuôi, từ lớp đầu vào (input) qua các lớp ẩn (hidden) và đến lớp đầu ra (ouput). Quá trình này được gọi là Forward Propagation (Lan truyền xuôi). Sau đây là những gì xảy ra trong giai đoạn này:

#### 2.1.1. Linear Transformation

Mỗi neuron là một lớp nhận đầu vào được nhân với các trọng số (weights) liên quan đến các kết nối. Kết quả được tính tổng và một bias được thêm vào tổng đó.

$$z = w_1x_1 + w_2x_2 + \cdots + w_nx_n + b$$

Trong đó:

- $w$ là các trọng số (weights)
- $x$ là dữ liệu đầu vào (inputs)

- $b$ là bias

### 2.1.2. Activation

Kết quả của linear transformation được đưa qua một **activation function** (hàm kích hoạt). Hàm kích hoạt rất quan trọng ví nó đưa tính phi tuyến tính vào hệ thống, giúp mạng có thể học được các đặc trưng phức tạp hơn. Một số hàm kích hoạt thường dùng là: ReLU, sigmoid và tanh.

## 2.2.Back Propagation (Lan truyền ngược)

Sau khi lan truyền xuôi, mạng sẽ đánh giá kết quả của nó bằng cách sử dụng các **loss function** (hàm mất mát), hàm này đo lường sự khác biệt giữa đầu ra thực tế và đầu ra dự đoán. Mục tiêu của huấn luyện là tối thiểu mất mát này. Đến đây **lan truyền ngược** bắt đầu phát huy tác dụng:

### 2.2.1. Loss Calculation

Mạng tính toán mất mát, cung cấp sự khác biệt của dự đoán so với thực tế. Hàm mất mát có thể thay đổi, lựa chọn phổ biến là MSE cho các nhiệm vụ hồi quy hoặc cross-entropy loss cho các bài toán phân loại.

### 2.2.2. Gradient Calculation

Mạng tính toán gradient của hàm mất mát đối với từng weight và bias trong mạng. Điều này liên quan đến việc áp dụng quy tắc chuỗi trong giải tích để tìm ra mức độ mà mỗi phần của lỗi đầu vào có thể được quy cho từng weight và bias.

### 2.2.3. Weights Update

Khi gradients được tính toán, weight và bias được cập nhật bằng các thuật toán tối ưu **(optimization algorithm)** như SGD. Trọng số được căn chỉnh theo hướng ngược lại của gradient để giảm thiểu mất mát. Kích thước của mỗi bước đi trong mỗi lần cập nhật được xác định bởi **learning rate**.

## 2.3. Iteration

Quá trình lan truyền xuôi, tính toán mất mát, lan truyền ngược và cập nhật trọng số được lặp lại nhiều lần. Qua đó, giảm thiểu mất mát và cải thiện độ chính xác của mạng trong các dự đoán.

Qua bước này, mạng neural có thể điều chỉnh các tham số của chúng để xấp xỉ tốt hơn các mốt quan hệ trọng dữ liệu, nhờ đó cải thiện hiệu suất của chúng trên các nhiện vụ như phân loại, hồi quy hoặc bất cứ mô hình dự đoán nào khác.

# II. Convolution Neural Network

## 1. Convolution Neural Network là gì?

Convolution Neural Network (Mạng neural tích chập), còn được gọi là ConvNets, là kiến trúc mạng neural lấy cảm hứng từ hệ thống thị giác của con người và được sử dụng rộng rãi trong các tác vụ thị giác máy tính. Chúng được thiết kế để xử lý dữ liệu giống như lưới có cấu trúc, đặc biệt là hình ảnh bằng cách nắm bắt các mối quan hệ không gian giữa các pixel.

## 2. Các thành phần chính của CNN

### 2.1.Input Layer

Lớp input nhận dữ liệu ảnh thô và chuyển nó qua mạng để xử lý. Trong **CNNs**, input thường là khối 3D (chiều rộng x chiều cao x chiều sâu).

- Lưu trữ các giá trị pixel của ảnh (vd: 32 x 32 x 3 cho các ảnh RGB).
- Giữ nguyên cấu trúc không gian của hình ảnh để trích xuất tính năng.

### 2.2.Convolutional Layer (Lớp tích chập)

**Lớp tích chập** chịu trách nhiệm trích xuất các đặt trưng quan trọng từ dữ liệu đầu vào. Nó áp dụng một loạt các bộ lọc có thể học được (**kernels**) trượt qua các hình ảnh và tính tích vô hướng (dot product) giữa trọng số của bộ lọc và các vùng ảnh hưởng tương ứng, từ đó tạo ra các bản đồ đặt trưng (**feature maps**)

- Sử dụng các bộ lọc nhỏ (vd: 2x2, 3x3, 5x5) để quét các ảnh đầu vào
- Tạo ra các bản đồ đặt trưng và nắm bắt các mẫu như canh, kết cấu và hình dạng.

### 2.3.Activation Layer

Các **lớp kích hoạt** đưa tính phi tuyến tính vào trong mạng bằng cách áp dụng một hàm kích hoạt theo từng phần tử (element-wise) lên đầu ra của lớp tích chập. Điều này giúp mô hình học được các mẫu phức tạp vượt ra ngoài các mối quan hệ tuyến tính.

- Các hàm kích hoạt phổ biến bao gồm **ReLU**, **Tanh** và **Leaky ReLU**.
- Hàm kích hoạt được áp dụng theo từng phần tử trên bản đồ đặt trưng
- Không làm thay đổi kích thước đầu ra

### 2.4.Pooling Layer

**Pooling Layer** được sử dụng để giả kích thước không gian của các bản đồ đặt trưng, giúp tăng tốc độ tính toán, giảm bộ nhớ sử dụng và hạn chế hiện tượng **overfitting**. Lớp này thường được đặt xen giữa các lớp tích chập trong mạng **CNN**.

- Các loại phổ biến bao gồm **Max Pooling** và **Average Pooling**
- Giảm chiều dài và chiều rộng nhưng giữ nguyên chiều sâu

### 2.5.Flattening

**Flattening** chuyển bản đồ đặt trưng đa chiều thành một vector một chiều sau các lớp tích chập và pooling. Vector này sau đó được chuyển qua lớp kết nối đầy đủ để thực hiện phân loại hoặc hồi quy

### 2.6.Fully Connected Layer

**Lớp kết nối đầy đủ** thực hiện suy luận ở mức cao dựa trên các đặc trưng đã được trích xuất và tạo ra các điểm số phân loại cuối cùng

### 2.7.Oupur Layer

Lớp đầu ra chuyển điểm số cuối cùng thành xác xuất sử dụng các hàm kích hoạt như **Sigmoid** (cho các bài toán phân loại nhị phân) hoặc **Sorfmax** (cho các bài toán phân loại đa lớp)

## 3. Convolutional Layers hoạt động như thế nào

- Một ma trận nhỏ được gọi là bộ lọc (kernel) trượt qua các hình ảnh để trích xuất các đặc trưng quan trọng.
- Tại mỗi vị trí, bộ lọc thực hiện phép nhân theo từng phần tử (element-wise multiplication) với một vùng ảnh tương ứng.
- Các giá trị sau khi nhân được cộng lại để tạo thành một giá trị đầu ra duy nhất.
- Quá trình này được lặp lại trên toàn bộ ảnh với một bước trượt (stride) xác định.
- Kết quả thu được là một ma trận mới gọi là **bản đồ đặt trưng (feature map)**, giúp làm nổi bật các mẫu đã được phát hiện.
- Nhiều bộ lọc khác nhau được sử dụng để trích xuất các đặc trưng khác nhau như cạnh, họa tiết và hình dạng.
- Quá trình này vẫn giữ được mối quan hệ không gian giữa các điểm ảnh, đồng thời giảm số lượng tham số cần học so với các lớp kết nối đầy đủ.
- Padding có thể được sử dụng để kiểm soát kích thước đầu ra và tránh mất thông tin ở vùng biên của ảnh

## 4. Triển khai một bộ phân loại ảnh dựa trên CNN sử dụng thư viện PyTorch

https://github.com/hoangvudinh0301/license-plate-ocr/blob/76674d23c4713d14c347a9cfac0d3e4428f1c403/SourceCode/cnn.ipynb

# III. Object Detection

## 1. Bounding Box Detection

### 1.1.Bounding Box là gì?

**Bouding box detection** là một nhiệm vụ cơ bản của thị giác máy tính nó cung cấp nhận diện và phát hiện các vật thể trong ảnh. Thay vì chỉ phân loại đối tượng, như

trong phân loại hình ảnh, bouding box detection cung cấp sự hiểu biết chi tiết hơn về không gian của từng đối tượng.

Xây dựng một mô hình phát dự đoán bouding box từ đầu băng các sử dụng **Pytorch** bao gồm tạo một mạng neural học các xác định vật thể trong ảnh. Nhiệm vụ này thường sử dụng mạng **CNN** để phát hiện cấu trúc không gian. Mô hình này được huấn luyện trên một bộ dataset có các bounding box được chú thích. Trong quá trình huấn luyện, mạng tinh chỉnh các tham số quả nó thông qua lan truyền ngược, giảm thiểu sự khác biệt giữa bounding box dự đoán và thực tế. Các thành phần chính bao gồm tiền xử lý hình ảnh, định nghĩa kiến trúc mạng neural với các đầu ra hồi quy cho tọa độ hộp và tối ưu hóa bằng một hàm mất mát.

### 1.2. Triển khai một ví dụ của Bounding Box Detection sử dụng thư viện PyTorch

https://github.com/hoangvudinh0301/license-plate-ocr/blob/76674d23c4713d14c347a9cfac0d3e4428f1c403/SourceCode/boundingBoxDetection.ipynb

## 2. Intersection over Union (IuO)

**Intersection over Union** là một tiêu trí đánh giá được sử dụng để đo độ chính xác của một mô hình phát hiện đối tượng trong một bộ dataset cụ thể. IoU chỉ đơn giản là một số liệu đánh giá, bất cứ bài toán nào cung cấp các bounding box dự đoán đều có thể được đánh giá bằng cách sử dụng IoU.

Để tính IoU cần có bouding box dự đoán và bounding box thât, công thức tính IoU:

$$IoU = \frac{Area\ of\ Overlap}{Area\ of\ Union}$$

Trong đó:

- $Area\ of\ Overlap$: là vùng chồng lên nhau giữa hai bounding box
- $Area\ of\ Union$: tổng diện tích của 2 box

Khi đó giá trị IoU sẽ nằm trong khoảng [0, 1]

Ví dụ code tình IoU

```python
import torch

def iou(box1, box2):
    # intersection
    x1 = torch.max(box1[0], box2[0])
    y1 = torch.max(box1[1], box2[1])
    x2 = torch.min(box1[2], box2[2])
    y2 = torch.min(box1[3], box2[3])

    intersection = torch.clamp(x2 - x1, min=0) * torch.clamp(y2 - y1, min=0)

    # area
    area1 = (box1[2] - box1[0]) * (box1[3] - box1[1])
    area2 = (box2[2] - box2[0]) * (box2[3] - box2[1])

    union = area1 + area2 - intersection

    return intersection / union
box1 = torch.tensor([50, 50, 200, 200])
box2 = torch.tensor([100, 100, 250, 250])

print(iou(box1, box2))
✓ [31] 77ms
```

```
tensor(0.2857)
```

## 3. Non-Max Suppression

**Non-Max Suppression** là một kỹ thuật sử lý hậu kì trong object detection, dùng để cải thiện các dự đoán thô được thực hiện bởi một mô hình. Khi mô hình phát hiện đối tượng phân tích hình ảnh, nó thường tạo ra các bounding box nằm chồng lên nhau với cùng một đối tượng nhưng có score khác nhau. Những dự đoán dư thừa xảy ra vì mô hình có thể phát hiện cùng một đặt trưng ở các tỉ lệ hoặc vị trí hơi khác nhau. NMS lọc đầu ra bằng cách chỉ giữ lại bounding box có độ chính xác cao nhất cho mỗi đối tượng và loại bỏ những bounding còn lại, đảm bảo rằng đầu ra cuối cùng sữ sạch sẽ, chính xác và không có bản sao.

### Quy trình của NMS

- Lọc: loại bỏ tất cả các bounding box có điểm tin cậy dưới ngưỡng xác định (ví dụ: 0.25) để loại bỏ ngay các dự đoán yếu.
- Sắp xếp: sắp xếp các box còn lại dựa theo thứ tự giảm dần của độ tin cậy.
- Chọn lựa: chọn box có điểm tin cậy cao nhất làm phát hiện hợp lệ.
- So sánh: so sánh box đã chọn này với tất cả các box còn lại sử dụng IoU
- Loại bỏ: nếu IoU giữa box đã chọn và một box khác vượt quá ngưỡng xác định trước (ví dụ: 0.45), hộp có điểm thấp hơn được coi là bản sao và bị loại bỏ.

- Lặp lại: lặp lại quá trình với box có điểm cao tiếp theo chưa bị loại bỏ hoặc chưa được chọn, cho đến khi tất cả các hộp được xử lý.

**Triển khai NMS với thư viện PyTorch**

```
import torch
import torchvision.ops as ops

boxes = torch.tensor(
    [
        [100, 100, 200, 200],
        [105, 105, 195, 195],
        [300, 300, 400, 400],
    ],
    dtype=torch.float32,
)

scores = torch.tensor([0.9, 0.8, 0.95], dtype=torch.float32)

keep_indices = ops.nms(boxes, scores, iou_threshold=0.5)

print(f"Indices to keep: {keep_indices.tolist()}")
✓ [30] 27ms
```

```
Indices to keep: [2, 0]
```